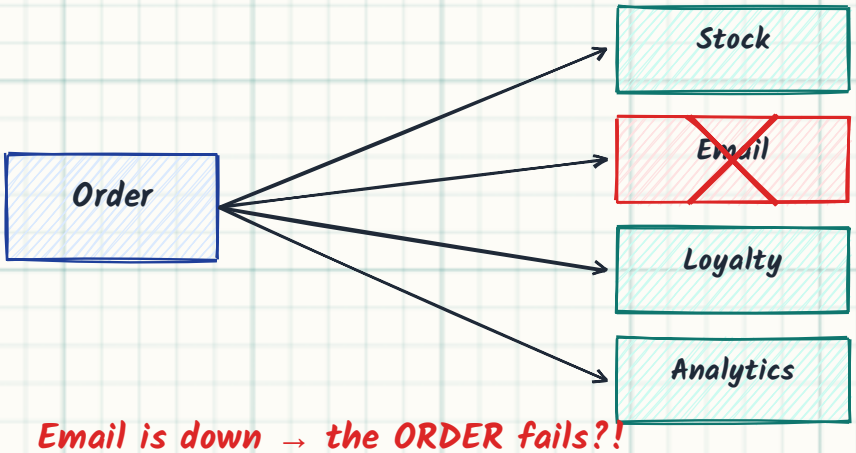
# Async Communication

Lesson 6 · events, queues & event-driven architecture

**In this lesson** → ① Sync fan-out problem ② Async way ③ Message broker ④ Queue vs topic ⑤ Commands vs events ⑥ Wins & challenges ⑦ Delivery guarantees ⑧ Sync or async? ⑨ Interview Q&A

**Running project:** ShopNest, our e-commerce app (users, products, orders, payments, notifications).

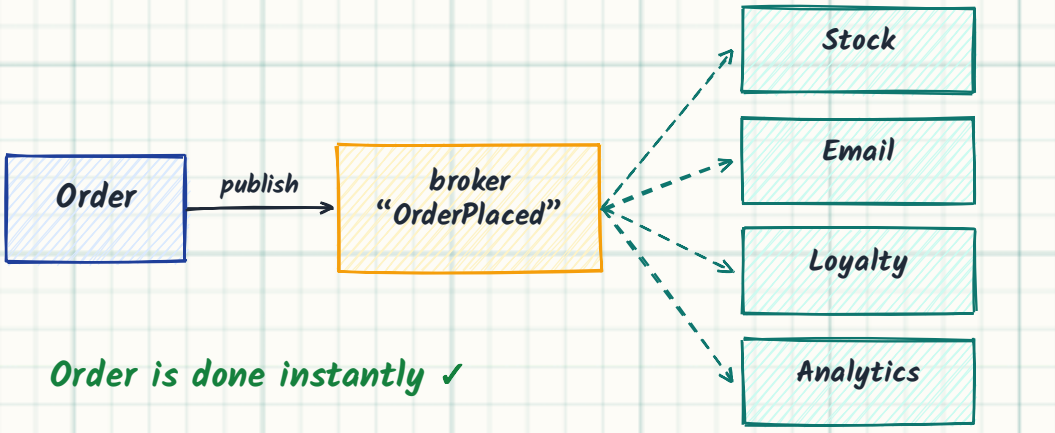
## ① The problem with sync fan-out




**Email is down → the ORDER fails?!**

- ✗ Order **waits** for 4 services → slow checkout
- ✗ Any one down (Email) → **order fails**
- ✗ Every new feature = Order service changes again

## ② The async way

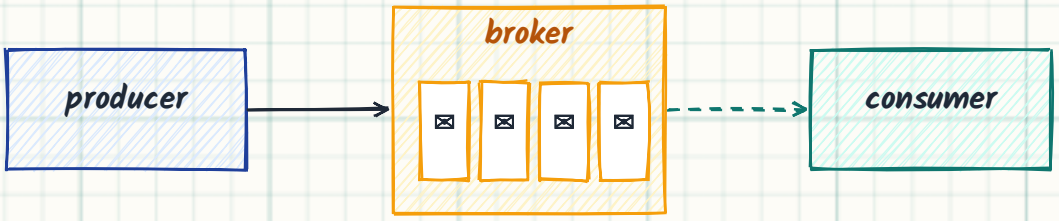



**Order is done instantly ✓**

Email down? it catches up later

> **Asynchronous communication** = the sender puts a message on a **broker** and moves on; receivers process it in their own time.

## ③ The message broker

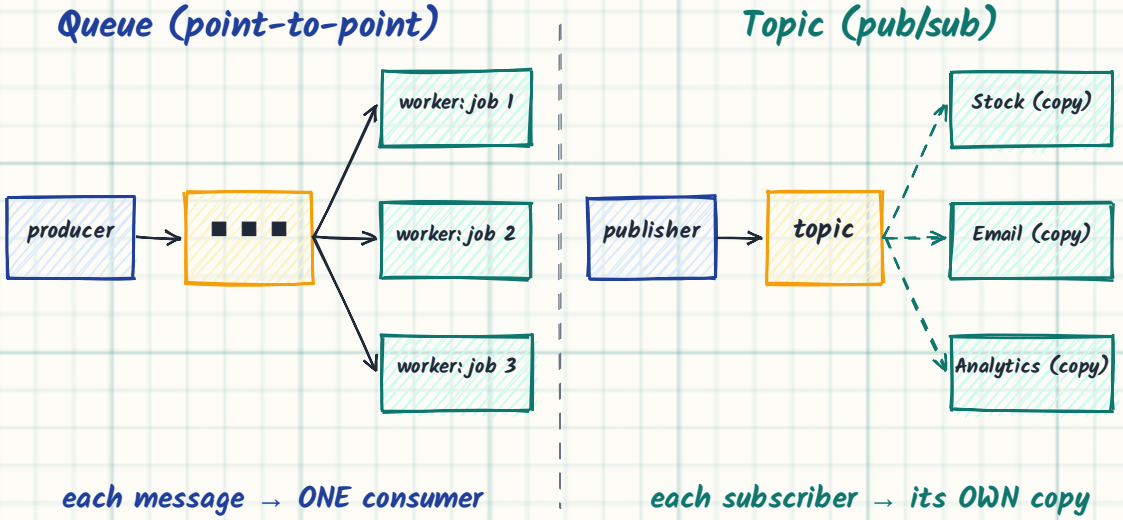



stores messages until they are handled

→ **Producer** sends → **broker** stores safely → **consumer** handles when ready

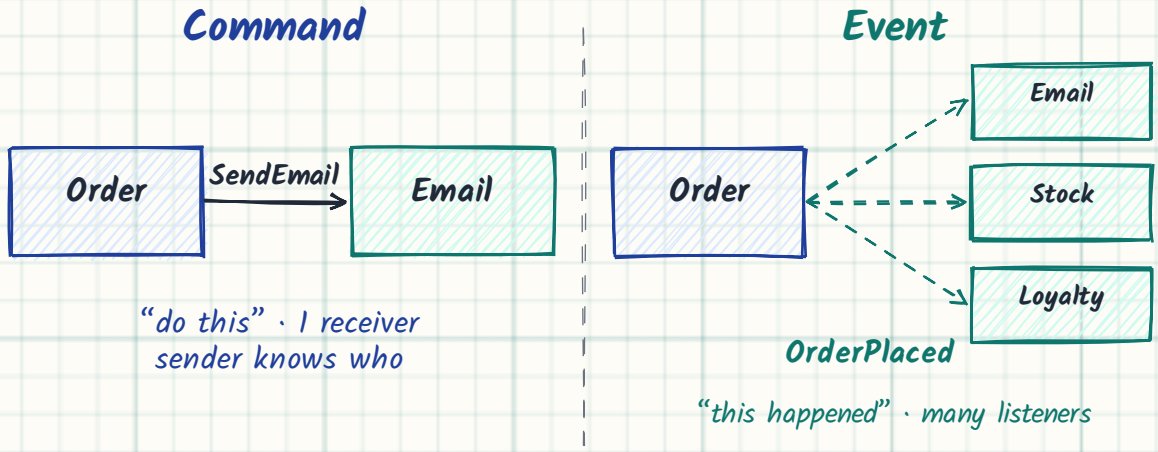
→ Examples: **RabbitMQ**, **Apache Kafka** (Lesson 7), **Amazon SQS / SNS**

## ④ Queue vs topic




| | Queue (point-to-point) | Topic (pub/sub) |
|---|---|---|
| **Delivery** | each message → ONE consumer | each subscriber gets a COPY |
| **Use for** | sharing work (send 1,000 emails) | broadcasting events (OrderPlaced) |
| **ShopNest** | email-jobs queue, 3 workers | order-events topic → Stock, Email, Analytics |

## 5 Commands vs events




| | Command | Event |
|---|---|---|
| **Meaning** | "do this" | "this happened" |
| **Naming** | imperative: SendEmail | past tense: OrderPlaced |
| **Receivers** | exactly one, known to sender | zero or many, unknown to publisher |
| **Coupling** | tighter | loosest |

**Key insight**
With events, a new feature (e.g. fraud check) = **add a listener**. The Order service does not change.

## 6 4 wins of async

- ✓ **Loose coupling**: producer and consumer need not be up at the same time
- ✓ **Absorbs spikes**: the broker buffers sale-day load
- ✓ **Easy to extend**: add consumers freely
- ✓ **Resilient**: a slow consumer doesn't block checkout

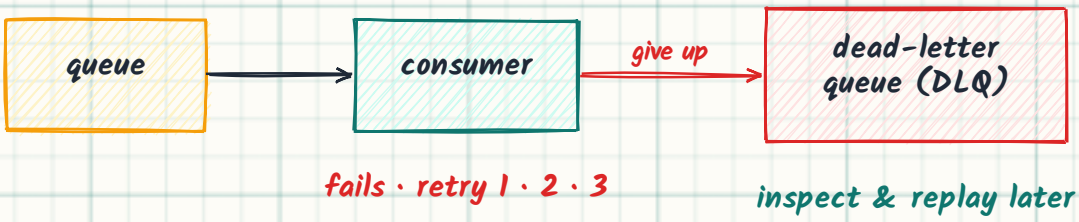
## 7 The challenges

| Challenge | What it means | How to handle |
|---|---|---|
| **Eventual consistency** | data catches up a moment later | design UI & flows to accept it |
| **Duplicates** | at-least-once → same message twice | **idempotent** consumers (store processed IDs) |
| **Ordering** | messages may arrive out of order | partition by key (Lesson 7), version fields |
| **Debugging** | one flow spans services & time | correlation IDs, tracing (Lesson 22) |
| **Poison messages** | a message that always fails | retries → dead-letter queue |

## 8 Delivery guarantees

| Guarantee | Lost? | Duplicated? |
|---|---|---|
| **At-most-once** | possible | never |
| **At-least-once** | never | possible |
| **Exactly-once** | never | never (hard, costly) |

*In practice*
**At-least-once + idempotent consumers** = effectively once.



## ⑨ Sync or async?

| Choose | When | ShopNest examples |
|---|---|---|
| **Sync** | the caller needs the answer now to continue | price check, login, "is it in stock?" |
| **Async** | side effects, slow work, others react | confirmation email, analytics, loyalty points |

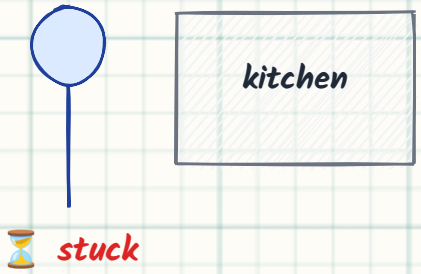
## ⑩ In code: Spring Boot + Kafka

```
// Order service: publish an event
kafkaTemplate.send("order-events", order.id(),
    new OrderPlaced(order.id(), order.customerId(), order.total()));

// Notification service: react to it
@KafkaListener(topics = "order-events", groupId = "notification")
void on(OrderPlaced e) {
    if (alreadyHandled(e.orderId())) return;   // idempotent
    email.sendConfirmation(e.orderId());
}
```

## ⑪ Restaurant analogy

**sync = waiter waits**




**async = order board**




waiter drops a ticket
and serves the next table

## 12 Interview Corner

**Q. Sync vs async communication?**

A. Sync waits for a response and couples services in time. Async sends a message and moves on: decoupled, but eventually consistent.

**Q. Queue vs topic?**

A. Queue: each message to one consumer (work sharing). Topic: every subscriber gets a copy (broadcast).

**Q. Command vs event?**

A. Command = "do this" to one known receiver. Event = "this happened", past tense, to any listeners.

**Q. How do you handle duplicate messages?**

A. Idempotent consumers, e.g. store processed message IDs and skip repeats.

**Q. What is a dead-letter queue?**

A. Where messages go after repeated failures, so they can be inspected and replayed without blocking others.

### ⚡ Revise in 60 seconds

- → Async = publish & move on
- → **Broker**: RabbitMQ, Kafka, SQS
- → Queue = 1 consumer · Topic = all
- → Event = past tense, many listeners
- → Wins: coupling, spikes, extend, resilience
- → **Eventual consistency**
- → At-least-once → **idempotency**
- → Poison message → **DLQ**

Next lesson → MS-07
**Apache Kafka** for microservices: topics, partitions, consumer groups.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**